

RECURSION TREE VISUALIZER – PROJECT REPORT

Design and Analysis of Algorithms (DAA)

Web-Based Visualization Tool for Divide-and-Conquer Algorithms

Algorithms Covered:

- Merge Sort
- Quick Sort
- Binary Search
- Closest Pair of Points

Introduction

Understanding recursive algorithms is one of the biggest challenges in learning Design and Analysis of Algorithms. Many students struggle to visualize how recursive calls are created and how they combine to produce the final result.

The Recursion Tree Visualizer is a web-based educational tool that helps students understand divide-and-conquer algorithms by displaying their recursive structure as an interactive tree. Each node represents a recursive call and edges represent the decomposition of the problem into smaller subproblems.

This project focuses on four fundamental divide-and-conquer algorithms: Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points problem.

Algorithm 1: Merge Sort

Merge Sort is a classic divide-and-conquer algorithm used for sorting arrays. It divides the array into two halves, recursively sorts each half, and merges them.

Steps:

1. Divide the array into two halves.
2. Recursively sort the left half.
3. Recursively sort the right half.
4. Merge the sorted halves.

Recurrence Relation:

$$T(n) = 2T(n/2) + O(n)$$

Time Complexity:

$$O(n \log n)$$

Algorithm 2: Quick Sort

Quick Sort selects a pivot element and partitions the array so that elements smaller than the pivot appear on the left and larger elements appear on the right.

Steps:

1. Select a pivot element.
2. Partition the array around the pivot.
3. Recursively apply Quick Sort to the left and right partitions.

Average Complexity:

$O(n \log n)$

Worst Case:

$O(n^2)$

Algorithm 3: Binary Search

Binary Search is an efficient algorithm used to find an element in a sorted array.

Steps:

1. Compare the target value with the middle element.
2. If equal, return the position.
3. If smaller, search the left half.
4. If larger, search the right half.

Time Complexity:

$O(\log n)$

Algorithm 4: Closest Pair of Points

The Closest Pair problem finds the pair of points with the smallest distance between them.

Divide and Conquer Approach:

1. Divide the points into two halves.
2. Recursively find closest pairs in both halves.
3. Check points across the dividing line.

Time Complexity:

$O(n \log n)$

System Architecture

The system is divided into three layers:

1. User Interface Layer

Contains controls such as algorithm selection, input fields, and navigation buttons.

2. Application Logic Layer

Implements algorithms and generates recursion tree structures.

3. Rendering Layer

Responsible for visualizing nodes, edges, and animations.

Dataset and Input Description

The application does not rely on external datasets.
All inputs are provided by the user at runtime.

Example Inputs:

Merge Sort: [5,3,8,2,9]

Quick Sort: [7,4,9,1,6]

Binary Search: [1,3,5,7,9], target = 7

Closest Pair: Set of coordinate points.

Conclusion

The Recursion Tree Visualizer demonstrates how divide-and-conquer algorithms work by visualizing recursive calls as tree structures.

This tool helps students:

- Understand recursion clearly
- Observe complexity patterns
- Interactively explore algorithms

Future enhancements include adding more algorithms such as Strassen Matrix Multiplication, Fibonacci recursion trees, and dynamic programming visualizations.

RECURSION TREE VISUALIZER – PROJECT REPORT

Design and Analysis of Algorithms (DAA)

Web-Based Visualization Tool for Divide-and-Conquer Algorithms

Algorithms Covered:

- Merge Sort
- Quick Sort
- Binary Search
- Closest Pair of Points

Introduction

Understanding recursive algorithms is one of the biggest challenges in learning Design and Analysis of Algorithms. Many students struggle to visualize how recursive calls are created and how they combine to produce the final result.

The Recursion Tree Visualizer is a web-based educational tool that helps students understand divide-and-conquer algorithms by displaying their recursive structure as an interactive tree. Each node represents a recursive call and edges represent the decomposition of the problem into smaller subproblems.

This project focuses on four fundamental divide-and-conquer algorithms: Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points problem.

Algorithm 1: Merge Sort

Merge Sort is a classic divide-and-conquer algorithm used for sorting arrays. It divides the array into two halves, recursively sorts each half, and merges them.

Steps:

1. Divide the array into two halves.
2. Recursively sort the left half.
3. Recursively sort the right half.
4. Merge the sorted halves.

Recurrence Relation:

$$T(n) = 2T(n/2) + O(n)$$

Time Complexity:

$$O(n \log n)$$

Algorithm 2: Quick Sort

Quick Sort selects a pivot element and partitions the array so that elements smaller than the pivot appear on the left and larger elements appear on the right.

Steps:

1. Select a pivot element.
2. Partition the array around the pivot.
3. Recursively apply Quick Sort to the left and right partitions.

Average Complexity:

$O(n \log n)$

Worst Case:

$O(n^2)$

Algorithm 3: Binary Search

Binary Search is an efficient algorithm used to find an element in a sorted array.

Steps:

1. Compare the target value with the middle element.
2. If equal, return the position.
3. If smaller, search the left half.
4. If larger, search the right half.

Time Complexity:

$O(\log n)$

Algorithm 4: Closest Pair of Points

The Closest Pair problem finds the pair of points with the smallest distance between them.

Divide and Conquer Approach:

1. Divide the points into two halves.
2. Recursively find closest pairs in both halves.
3. Check points across the dividing line.

Time Complexity:

$O(n \log n)$

System Architecture

The system is divided into three layers:

1. User Interface Layer

Contains controls such as algorithm selection, input fields, and navigation buttons.

2. Application Logic Layer

Implements algorithms and generates recursion tree structures.

3. Rendering Layer

Responsible for visualizing nodes, edges, and animations.

Dataset and Input Description

The application does not rely on external datasets.
All inputs are provided by the user at runtime.

Example Inputs:

Merge Sort: [5,3,8,2,9]

Quick Sort: [7,4,9,1,6]

Binary Search: [1,3,5,7,9], target = 7

Closest Pair: Set of coordinate points.

Conclusion

The Recursion Tree Visualizer demonstrates how divide-and-conquer algorithms work by visualizing recursive calls as tree structures.

This tool helps students:

- Understand recursion clearly
- Observe complexity patterns
- Interactively explore algorithms

Future enhancements include adding more algorithms such as Strassen Matrix Multiplication, Fibonacci recursion trees, and dynamic programming visualizations.

RECURSION TREE VISUALIZER – PROJECT REPORT

Design and Analysis of Algorithms (DAA)

Web-Based Visualization Tool for Divide-and-Conquer Algorithms

Algorithms Covered:

- Merge Sort
- Quick Sort
- Binary Search
- Closest Pair of Points

Introduction

Understanding recursive algorithms is one of the biggest challenges in learning Design and Analysis of Algorithms. Many students struggle to visualize how recursive calls are created and how they combine to produce the final result.

The Recursion Tree Visualizer is a web-based educational tool that helps students understand divide-and-conquer algorithms by displaying their recursive structure as an interactive tree. Each node represents a recursive call and edges represent the decomposition of the problem into smaller subproblems.

This project focuses on four fundamental divide-and-conquer algorithms: Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points problem.

Algorithm 1: Merge Sort

Merge Sort is a classic divide-and-conquer algorithm used for sorting arrays. It divides the array into two halves, recursively sorts each half, and merges them.

Steps:

1. Divide the array into two halves.
2. Recursively sort the left half.
3. Recursively sort the right half.
4. Merge the sorted halves.

Recurrence Relation:

$$T(n) = 2T(n/2) + O(n)$$

Time Complexity:

$$O(n \log n)$$

Algorithm 2: Quick Sort

Quick Sort selects a pivot element and partitions the array so that elements smaller than the pivot appear on the left and larger elements appear on the right.

Steps:

1. Select a pivot element.
2. Partition the array around the pivot.
3. Recursively apply Quick Sort to the left and right partitions.

Average Complexity:

$O(n \log n)$

Worst Case:

$O(n^2)$

Algorithm 3: Binary Search

Binary Search is an efficient algorithm used to find an element in a sorted array.

Steps:

1. Compare the target value with the middle element.
2. If equal, return the position.
3. If smaller, search the left half.
4. If larger, search the right half.

Time Complexity:

$O(\log n)$

Algorithm 4: Closest Pair of Points

The Closest Pair problem finds the pair of points with the smallest distance between them.

Divide and Conquer Approach:

1. Divide the points into two halves.
2. Recursively find closest pairs in both halves.
3. Check points across the dividing line.

Time Complexity:

$O(n \log n)$

System Architecture

The system is divided into three layers:

1. User Interface Layer

Contains controls such as algorithm selection, input fields, and navigation buttons.

2. Application Logic Layer

Implements algorithms and generates recursion tree structures.

3. Rendering Layer

Responsible for visualizing nodes, edges, and animations.

Dataset and Input Description

The application does not rely on external datasets.
All inputs are provided by the user at runtime.

Example Inputs:

Merge Sort: [5,3,8,2,9]

Quick Sort: [7,4,9,1,6]

Binary Search: [1,3,5,7,9], target = 7

Closest Pair: Set of coordinate points.

Conclusion

The Recursion Tree Visualizer demonstrates how divide-and-conquer algorithms work by visualizing recursive calls as tree structures.

This tool helps students:

- Understand recursion clearly
- Observe complexity patterns
- Interactively explore algorithms

Future enhancements include adding more algorithms such as Strassen Matrix Multiplication, Fibonacci recursion trees, and dynamic programming visualizations.

RECURSION TREE VISUALIZER – PROJECT REPORT

Design and Analysis of Algorithms (DAA)

Web-Based Visualization Tool for Divide-and-Conquer Algorithms

Algorithms Covered:

- Merge Sort
- Quick Sort
- Binary Search
- Closest Pair of Points

Introduction

Understanding recursive algorithms is one of the biggest challenges in learning Design and Analysis of Algorithms. Many students struggle to visualize how recursive calls are created and how they combine to produce the final result.

The Recursion Tree Visualizer is a web-based educational tool that helps students understand divide-and-conquer algorithms by displaying their recursive structure as an interactive tree. Each node represents a recursive call and edges represent the decomposition of the problem into smaller subproblems.

This project focuses on four fundamental divide-and-conquer algorithms: Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points problem.

Algorithm 1: Merge Sort

Merge Sort is a classic divide-and-conquer algorithm used for sorting arrays. It divides the array into two halves, recursively sorts each half, and merges them.

Steps:

1. Divide the array into two halves.
2. Recursively sort the left half.
3. Recursively sort the right half.
4. Merge the sorted halves.

Recurrence Relation:

$$T(n) = 2T(n/2) + O(n)$$

Time Complexity:

$$O(n \log n)$$